

1-2-1 アルゴリズム1_基本的なデータ構造

目次 Contents

1 配列とリスト

(1) 配列

① 1次元配列

② 2次元配列

(2) リスト

① 単方向線形リスト

② 単方向環状リスト

③ 例題

2 スタックとキュー

(1) スタック

(2) キュー

(3) 例題

1 配列とリスト

データ構造とは、**データをコンピュータの中で効果的に扱うため、一定の形式に系統立てて格納するときの形式のこと**です。プログラムを書く際にはどのような処理を行うのかを事前に考え、その処理にとって最適なデータ構造を選択する必要があります。

配列とリストは、いずれもデータ構造です。データの並べ方、管理方法について似ている部分もあれば相違点もあります。それぞれの特徴を押さえていきましょう。

(1) 配列

配列とは、**数値のみを並べたデータ構造**です。配列には、以下の特徴があります。

配列の特徴

- ・添字を使ってデータの順序を表現します。
- ・メモリーの連続した領域にデータの順序通りに格納されます。
- ・後からデータを追加・削除する場合、そこから後ろのデータをずらして書き直さなければなりません。

配列には、1次元配列や多次元配列（主に2次元）などがあります。

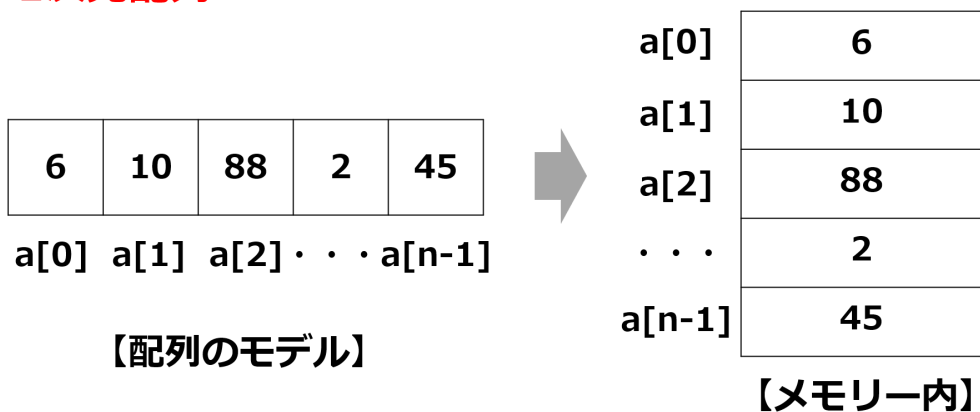
① 1次元配列

1次元配列は、添字の数が**1つの配列**です。**線形配列**とも呼ばれます。

※添字は大抵0からスタートします。

1次元配列のモデルは、以下の図のような形になります。実際にメモリーに格納される場合は、添字の昇順に縦に並びます。

1次元配列

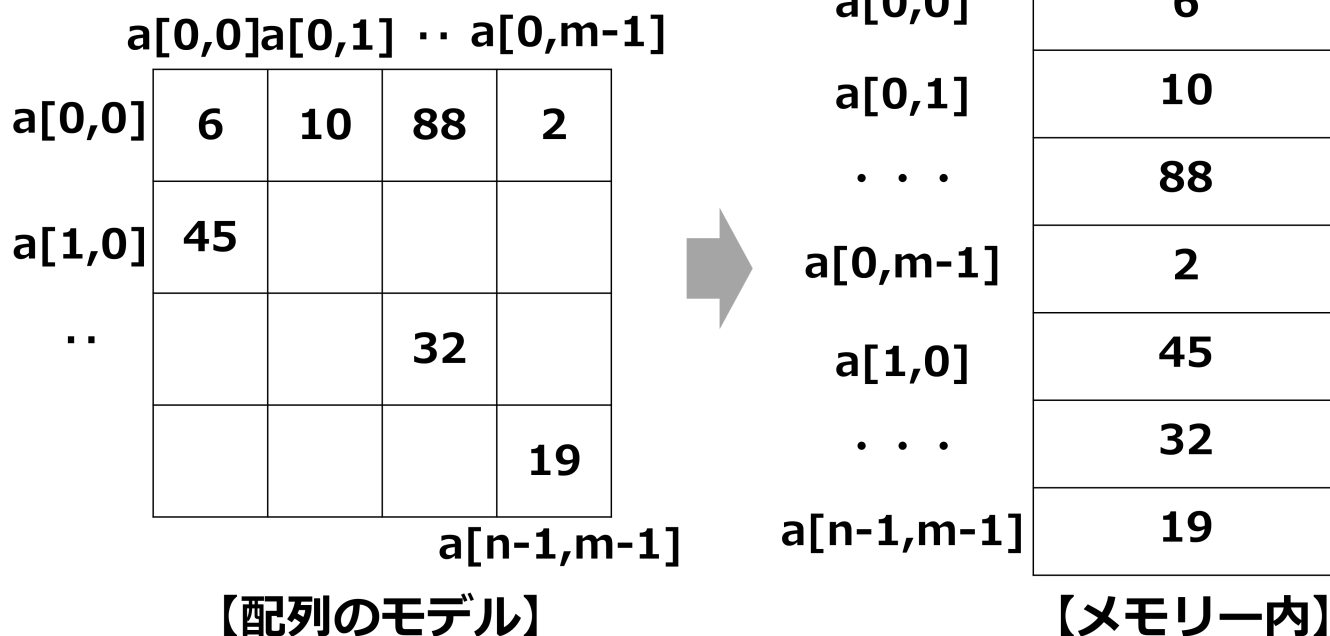


② 2次元配列

2次元配列は、添字の数が**2つの配列**です。メモリー上の並びは、1次元配列と同じ形になります。

添字の数を増やすことで、**n次元配列（多次元配列）**を作ることにも可能です。

2次元配列



【補足】 ガウスの計算法

1から100まで並んだ100個の数字の和を求める場合に、端と端を順番に足していくというものです。単純に1から100までを足していくのではなく、1と100の和である101が50個ある、という考え方です。

例えば0から99までの和を求める場合には、0と99の和が50個あると考えられるため、 $99 \times 50 = 4,950$ になります。

配列の数を問われる問題などで、活用すると早く計算をすることができます。

(2) リスト

配列では、後からデータを追加・削除する場合、そこから後ろのデータをずらして書き直さなければなりません。つまり、要素の追加や削除が非効率というデメリットがあります。

それを踏まえて追加や削除を効率良く行うことが出来るように改良したデータ構造がリストです。

リスト（連結リスト）とは、**データのつながりに意味を持たせたデータ構造**です。リストには、以下の特徴があります。

リストの特徴

- ・アドレスにより、データの位置が示されます。
- ・次のデータの位置を示すポインタを持たせることで、データのつながりを表現します。従って、データは物理的に順番に並んでいる必要はありません。並び替えも容易にできます。

アドレス

データ	ポインタ
-----	------

リストは、その実現の仕方によっていくつか細分化することができます。

リストの種類

・要素が $A \Rightarrow B \Rightarrow C \Rightarrow D$ のように、次へのポインタのみを持つリストを**単方向リスト**と呼びます。 $A \Leftrightarrow B \Leftrightarrow C \Leftrightarrow D$ のように、前後へのポインタを両方持つリストを**双方向リスト**と呼びます。

・リストに端（先頭と末尾）があるものを**線形リスト**、最初の要素と最後の要素も連結してループ状の形になっているものを**環状リスト**と呼びます。

これら2つずつの組み合わせによって、以下の4つのタイプがあります。

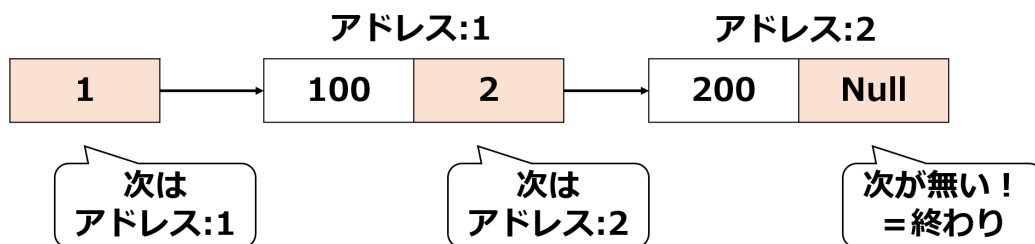
- ・ 単方向線形リスト
- ・ 単方向環状リスト
- ・ 双方向線形リスト

- 双方向環状リスト

では、頻出の単方向線形リストを詳しく見ていきましょう。

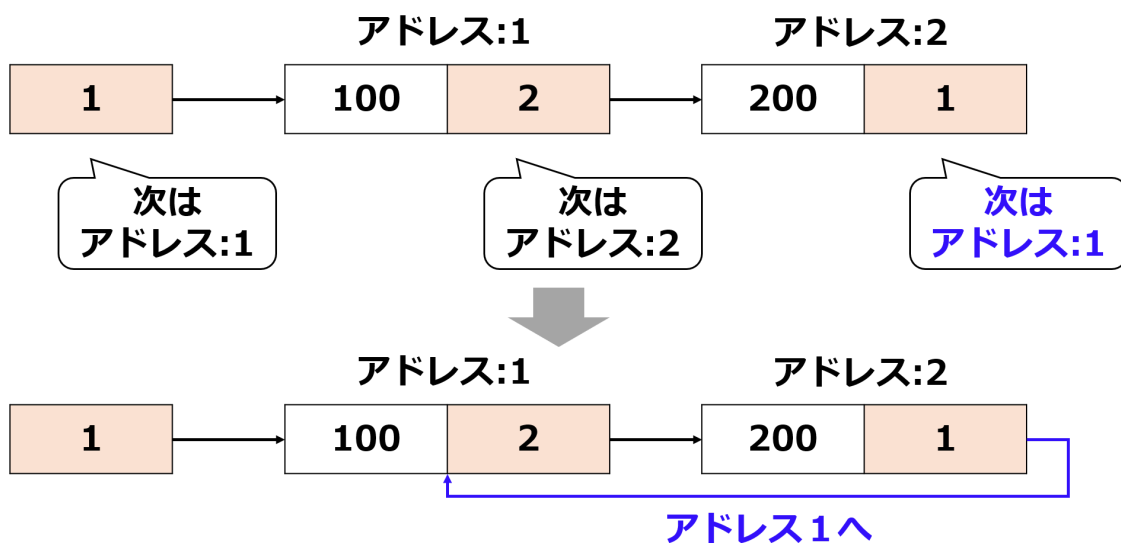
① 単方向線形リスト

単方向線形リストとは、**次へのポインタのみを持ち、かつリストに端（先頭と末尾）があるリスト**です。最も一般的なリストになります。



② 単方向環状リスト

単方向環状リストとは、**次へのポインタのみを持ち、かつリストがループ状に連結しているもの**です。以下の図のように、終わりが無いリストになります。



では、実際にリストの問題にチャレンジしてみましょう。

③ 例題

ポインタを用いた線形リストの特徴のうち、適切なものはどれか。

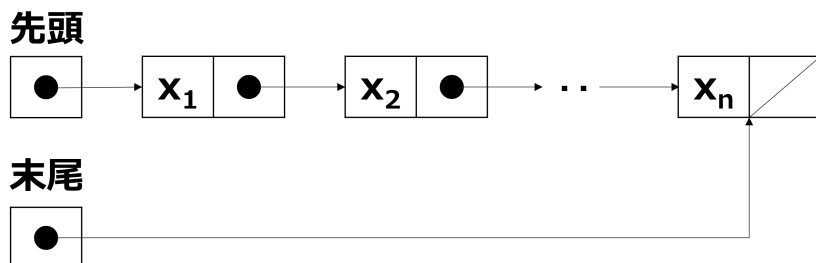
- ア 先頭の要素を根としたn分木で、先頭以外の要素は全て先頭の要素の子である。
- イ 配列を用いた場合と比較して、2分探索を効率的に行うことが可能である。
- ウ ポインタから次の要素を求めるためにハッシュ関数を用いる。

エ ポインタによって指定されている要素の後ろに、新たな要素を追加する計算量は、要素の個数や位置によらず一定である。

(応用情報技術者試験 令和2年度秋期 午前問5)

解答：エ

ポインタを用いた線形リストは、例えば以下ようになります。



これを踏まえて、選択肢を1つずつ確認していきましょう。

ア 先頭の要素を根としたn分木で、先頭以外の要素は全て先頭の要素の子である。

→これは、木構造の説明になります。よって、当てはまりません。

イ 配列を用いた場合と比較して、2分探索を効率的に行うことが可能である。

→これは、2分探索木についての説明となっています。そのため、正しい選択肢とはいえません。

また、線形リストはデータを先頭から順に探索していく構造であるため、2分探索を効率的に行うことが可能ではありません。

ウ ポインタから次の要素を求めるためにハッシュ関数を用いる。

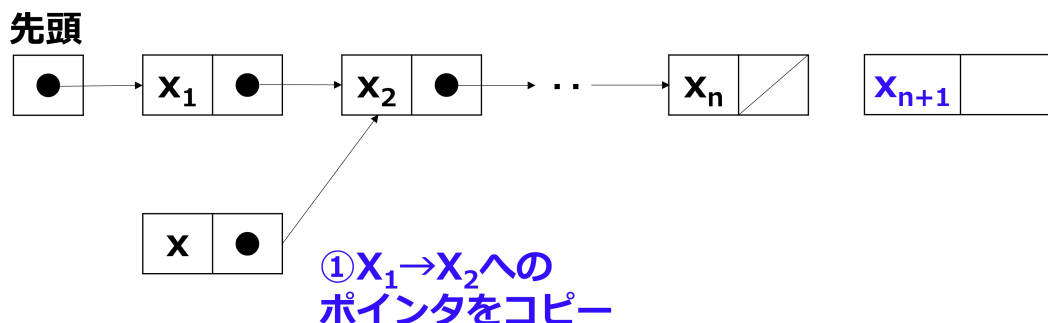
→ポインタは、次のデータの位置を示します。次の要素を得るには、ポインタが指し示すメモリ上のデータを読み込みます。従って、ハッシュ関数は使いません。

エ ポインタによって指定されている要素の後ろに、新たな要素を追加する計算量は、要素の個数や位置によらず一定である。

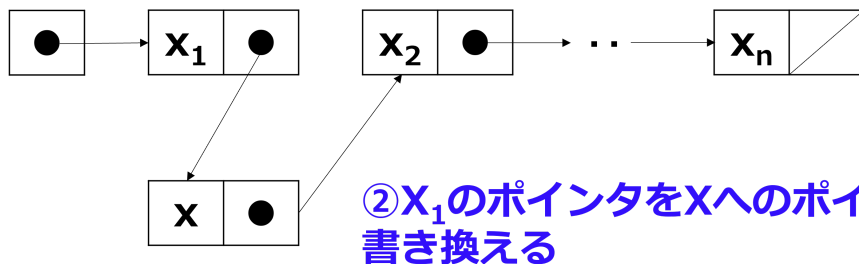
→ポインタによって指定されている要素の後ろに、新たな要素を追加する場合を例を使って考えてみましょう。

例) X_1 の後ろに要素 X を追加する場合

X_1 の後ろに要素 X を追加する場合、まずは $X_1 \rightarrow X_2$ へのポインタをコピーします。



次に、 $X_1 \rightarrow X_2$ のポインタを、 $X_1 \rightarrow X$ へのポインタに書き換えます。

先頭

これは、他の要素の後ろに要素を追加する場合も同じ手順になります。つまり、ポインタによって指定されている要素の後ろに新たな要素を追加する計算量は、要素の個数や位置によらず一定になります。

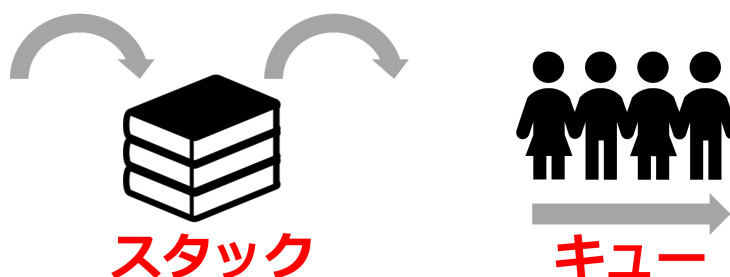
以上より、正解は**I**です。

2 スタックとキュー

スタックとキューも、同じくデータ構造ではありますが、データの取り出す構造にそれぞれ違いがあります。

スタックは、積み上げられた本のイメージです。**最後に積み上げたものから取り出して処理するデータ構造**になっています。

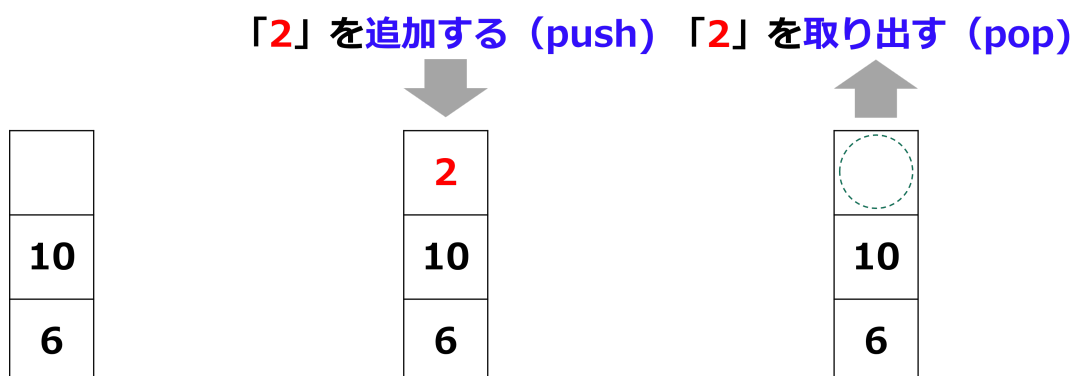
キューは、英語で「列」という意味ですが、その名の通りで**先に並んだものから順番に取り出して処理するデータ構造**になっています。



(1) スタック

スタックでは、最後に追加した要素を最初に取り出します。これを**LIFO (Last-In-First-Out)**と言います。

スタックにおけるデータの格納と取り出しの順番を配列で表現すると、以下の図のようになります。

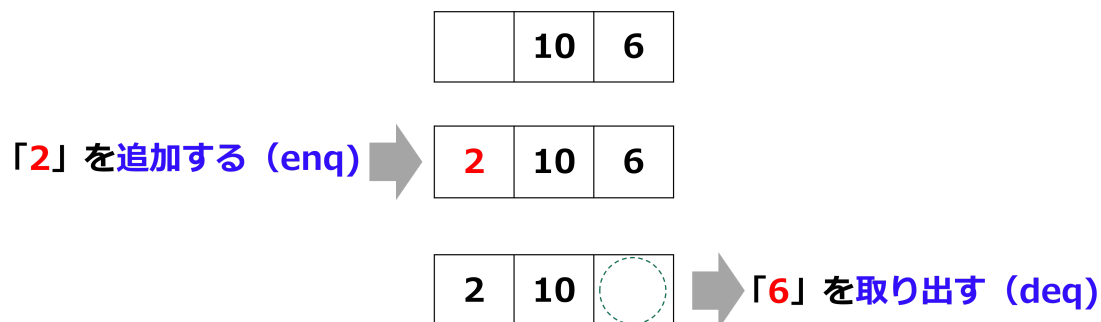


データを追加することをpush、データを取り出すことをpop(pull)と言います。

(2) キュー

キューでは、最初に追加した要素を最初に取り出します。これを**FIFO (First-In-First-Out)**と言います。

キューにおけるデータの格納と取り出しの順番を配列で表現すると、以下の図のようになります。



(3) 例題

スタックとキューの二つのデータ構造がある。次の手続を順に実行した場合、変数xに代入されるデータはどれか。ここで、

データyをスタックに挿入することを push(y),

スタックからデータを取り出すことを pop(),

データyをキューに挿入することを enq(y),

キューからデータを取り出すことを deq(),

とそれぞれ表す。

push(a)

push(b)

enq(pop())

enq(c)

push(d)

push(deq())

x ← pop()

ア a イ b ウ c エ d

(応用情報技術者試験 平成16年度春期 午前問10)

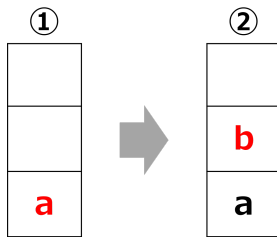
解答：イ

まず、1～2行目を見ていきましょう。

①push(a) : スタックに「a」を追加する

②push(b) : スタックに「b」を追加する

この2行を実行すると、スタックは以下の図のようになります。

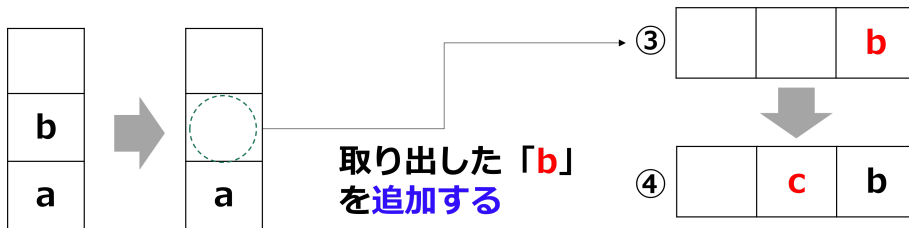


次に、3～4行目を実行しましょう。

③enq(pop()) : キューに「スタックから取り出した値」を追加する

④enq(c) : キューに「c」を追加する

この2行を実行すると、スタックとキューは以下の図のようになります。

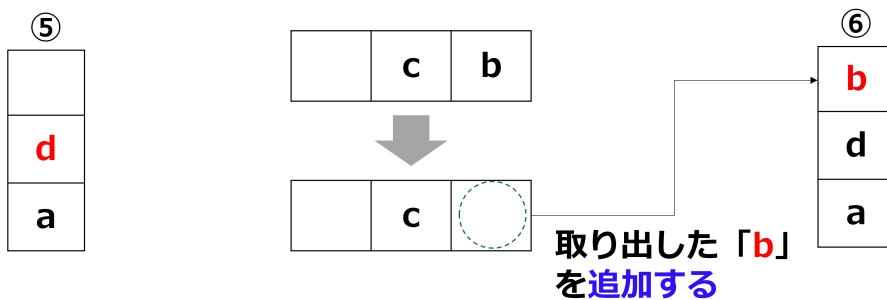


次に、5～6行目を実行しましょう。

⑤push(d) : スタックに「d」を追加する

⑥push(deq()) : スタックに「キューから取り出した値」を追加する

この2行を実行すると、スタックとキューは以下の図のようになります。



最後に、7行目を実行しましょう。

⑦x ← pop() : xに「スタックから取り出した値」を代入する

よって、xはスタックから取り出した値であるbになります。

以上より、正解は**イ**になります。